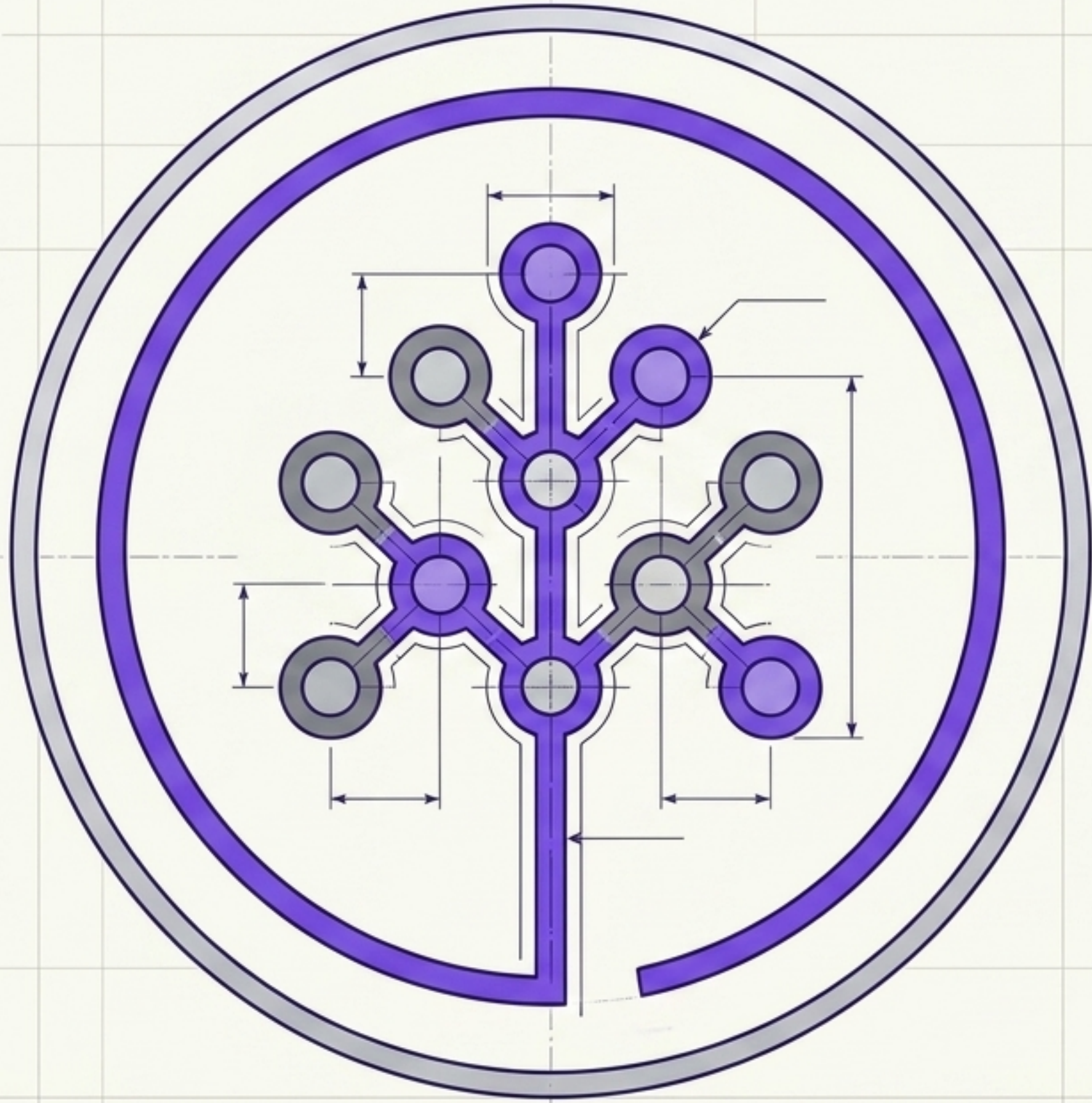
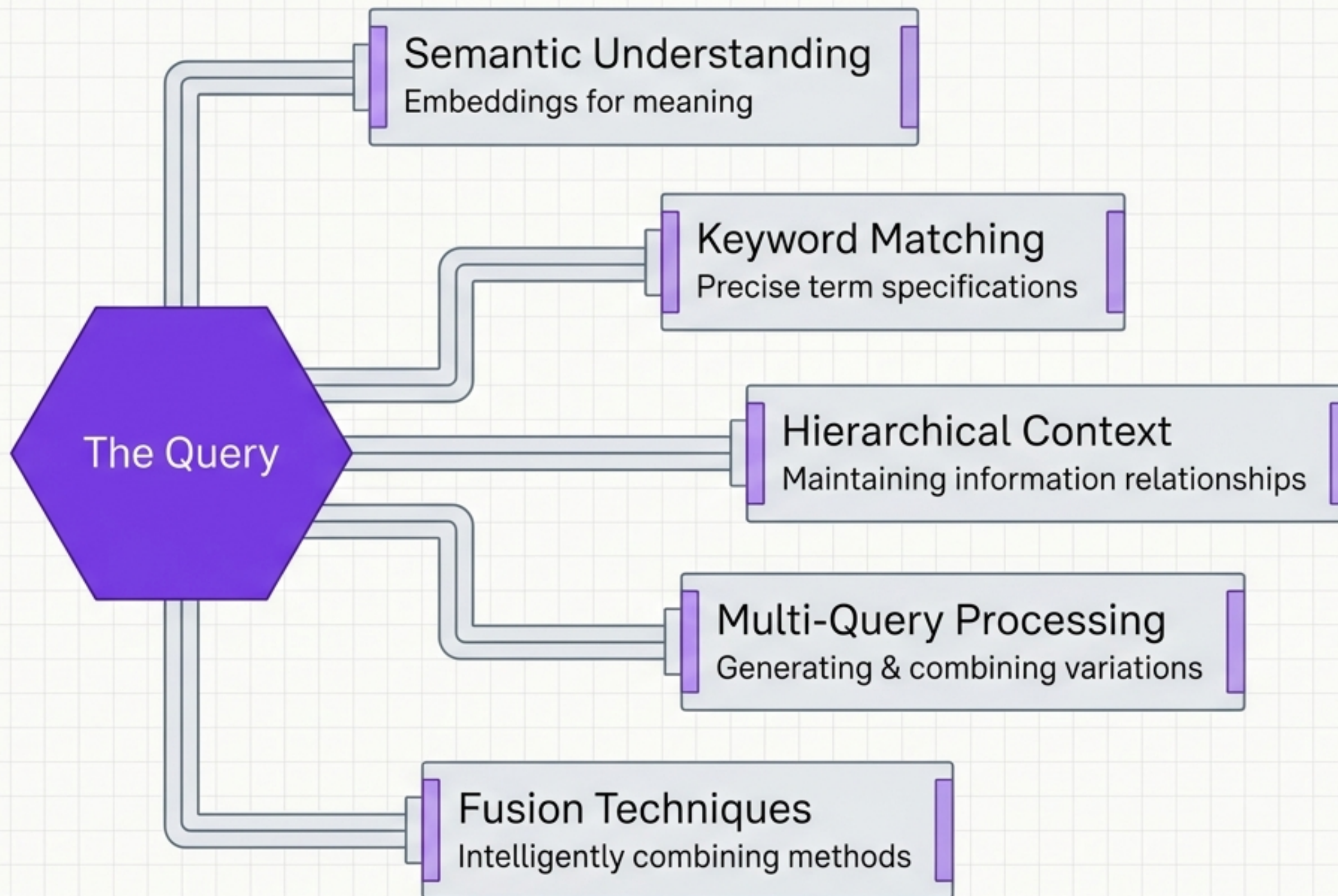




Advanced Retrievers for RAG

The Architect's Playbook for LlamaIndex and LangChain

Beyond Simple Similarity

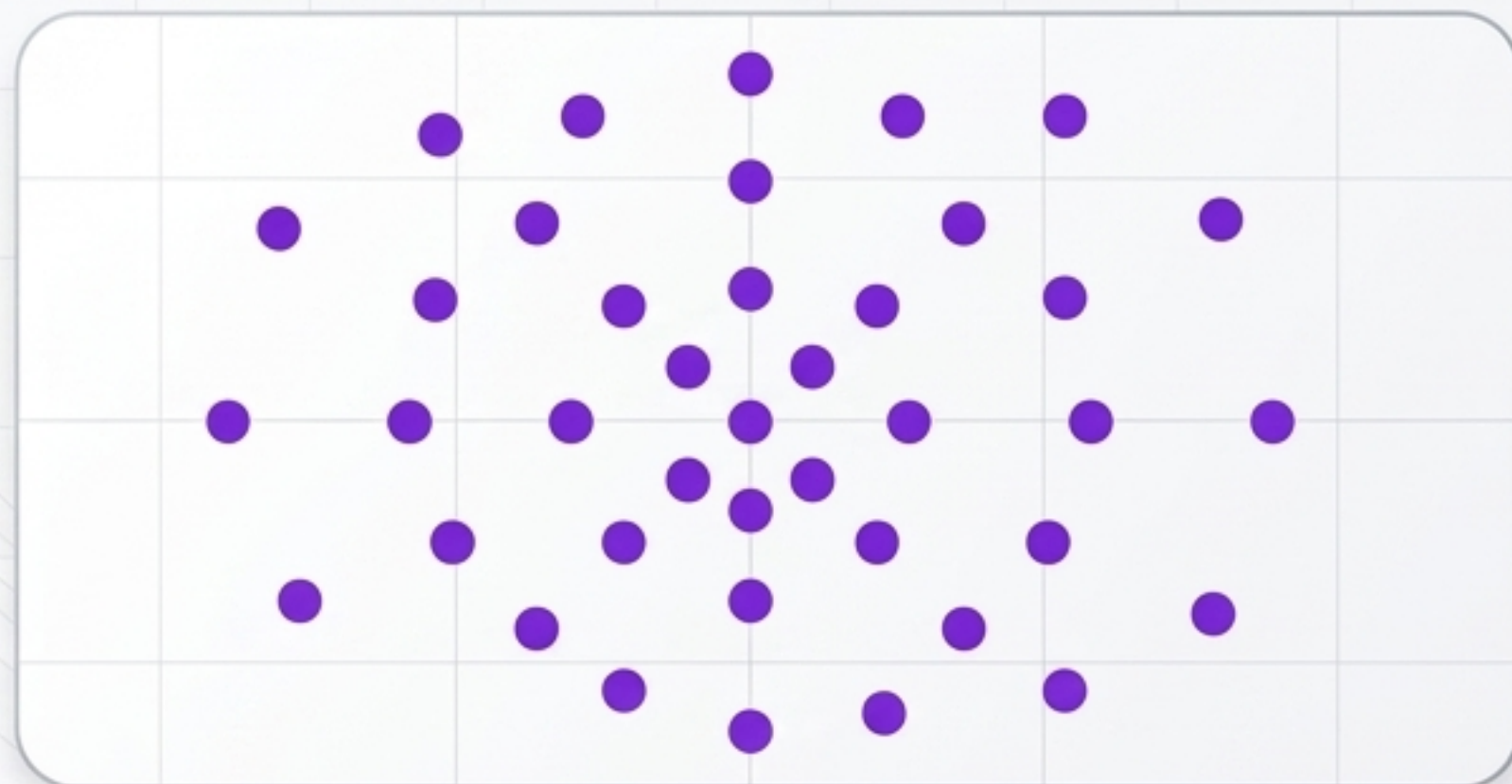


The Guiding Principle: Maximum Marginal Relevance (MMR)

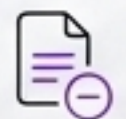
Without MMR



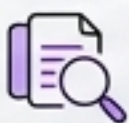
With MMR



Purpose: Balance relevance and diversity of retrieved results.



Method: Select documents highly relevant to the query AND minimally similar to previously selected documents.



Benefit: Avoid redundancy and ensure comprehensive coverage of different query aspects.

The LlamaIndex Arsenal: Core Index Architectures

VectorStoreIndex

Function

Stores vector embeddings for each document chunk.

Best For

Semantic retrieval based on meaning.

Usage

Commonly used in LLM pipelines and RAG applications.

DocumentSummaryIndex

Function

Generates and stores summaries of documents at indexing time.

Best For

Large documents whose meanings are lost by chunking (exceeds context windows).

Usage

Returns original documents, using summaries to retrieve based on the semantic meaning of the entire text.

KeywordTableIndex

Function

Extracts keywords and maps to content chunks.

Best For

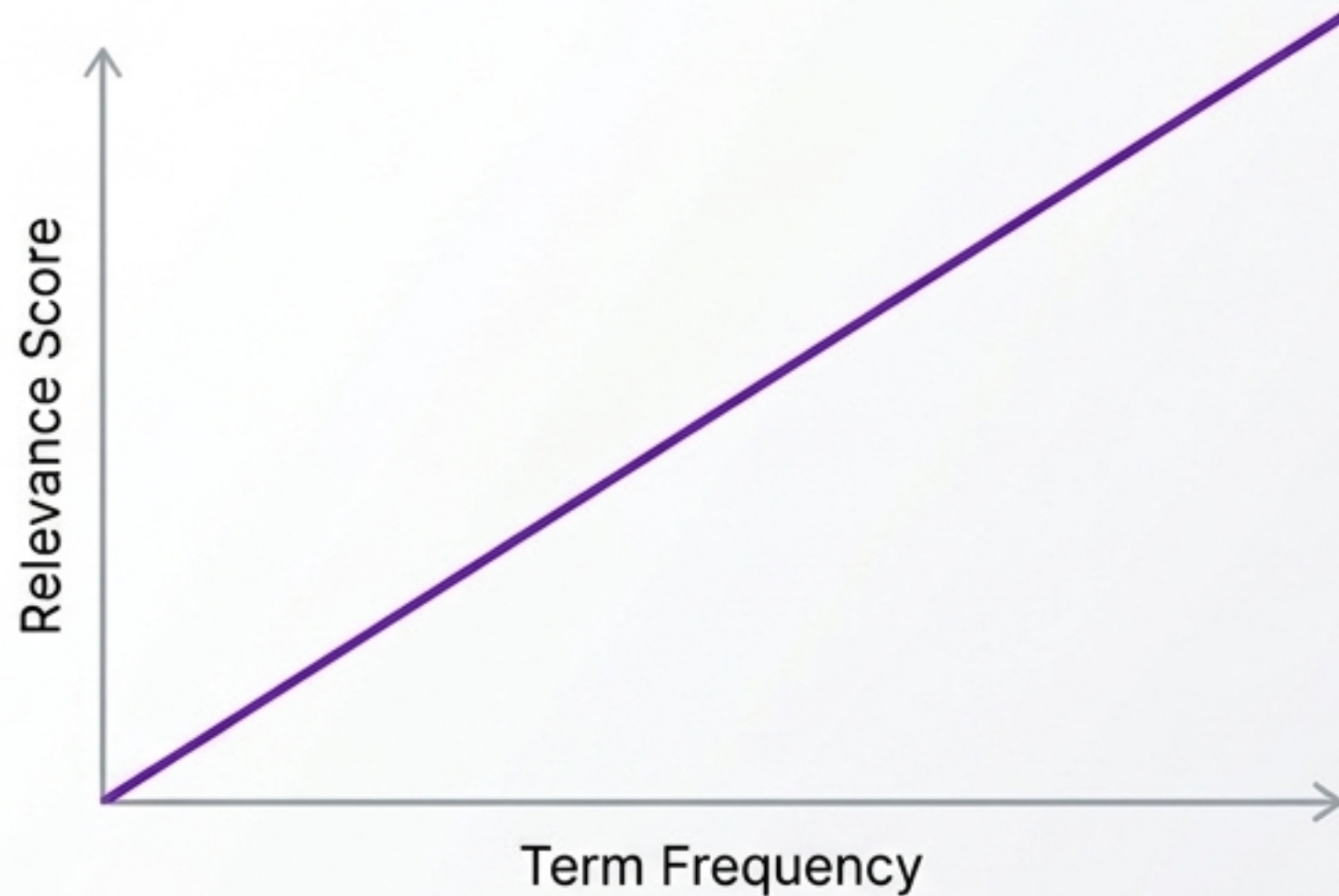
Exact keyword matching.

Usage

Rule-based or hybrid search scenarios requiring precise term matching.

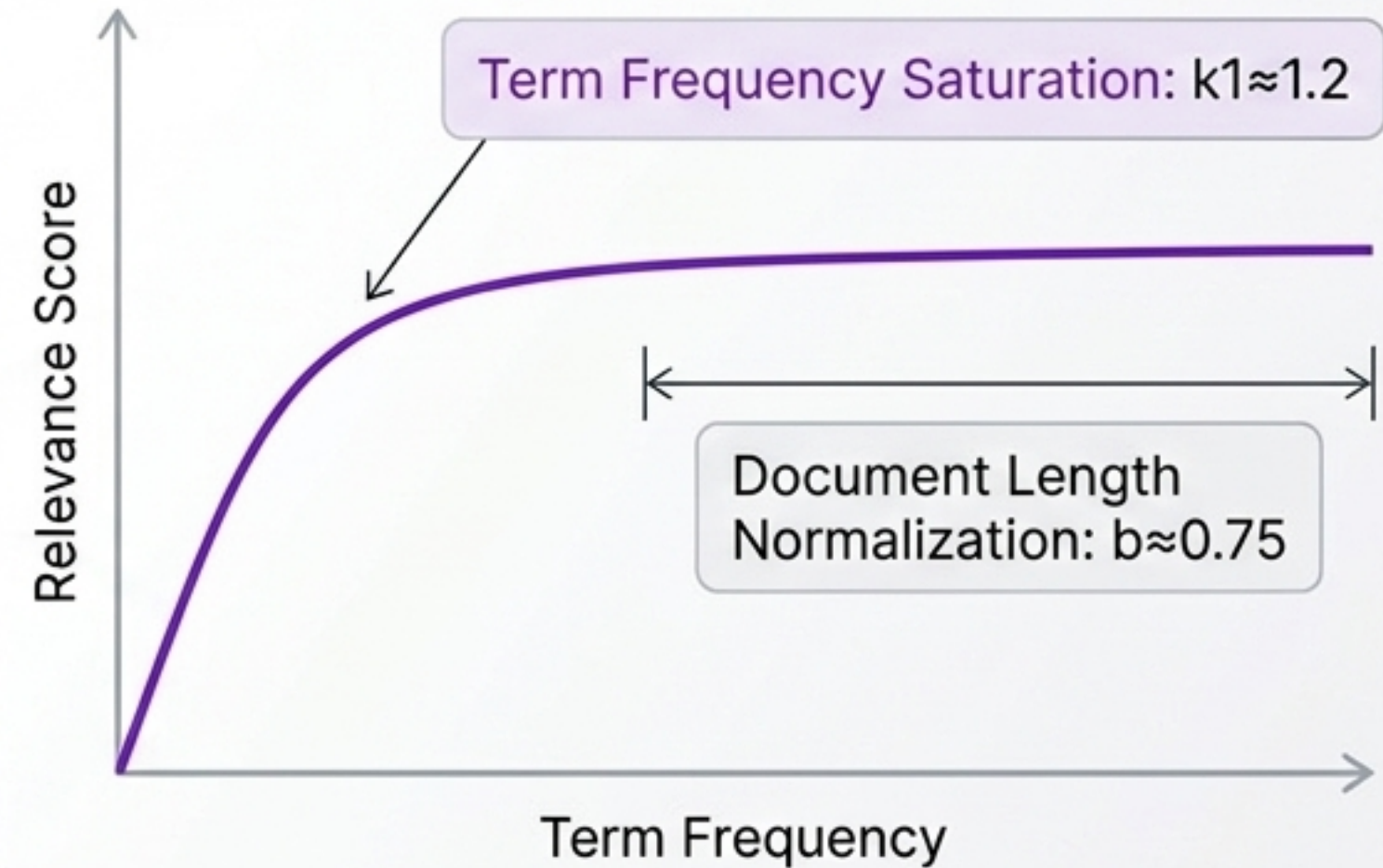
BM25 Retriever: Advanced Keyword Precision

TF-IDF Foundation



TF-IDF limitation: Term frequency scales endlessly.

BM25 Improvements

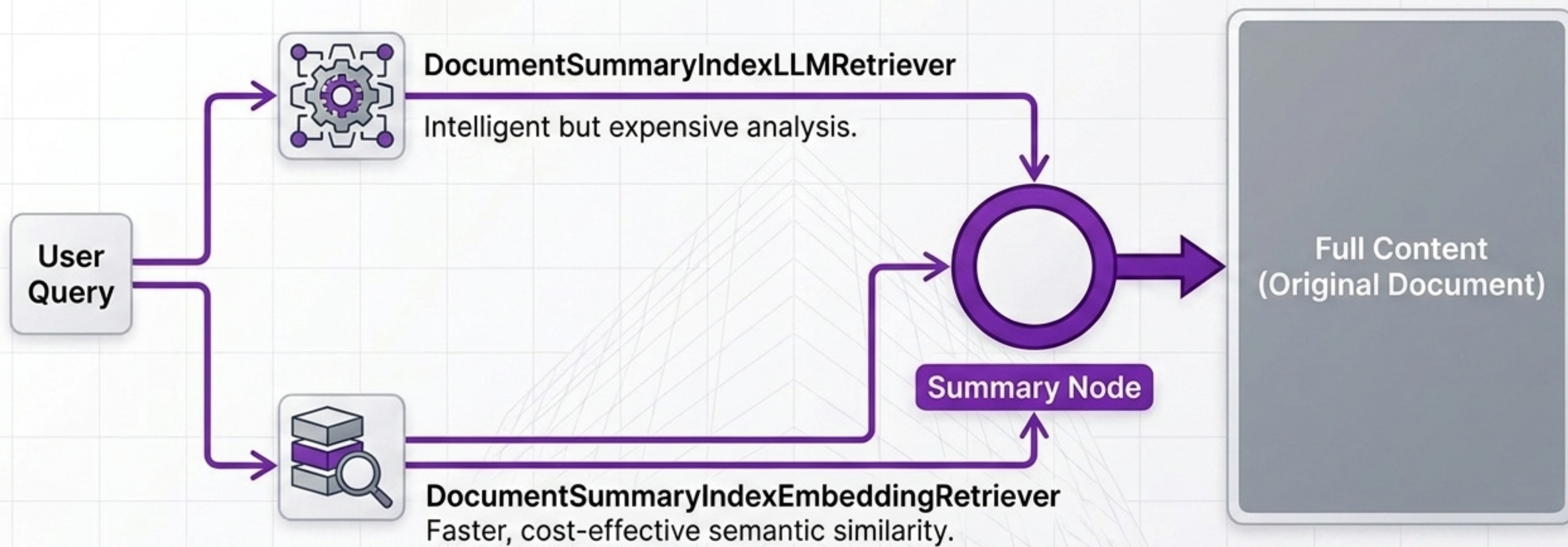


BM25 Mechanism: Reduces impact of repeated terms (saturation) and mathematically adjusts for document length to prevent long-document bias.

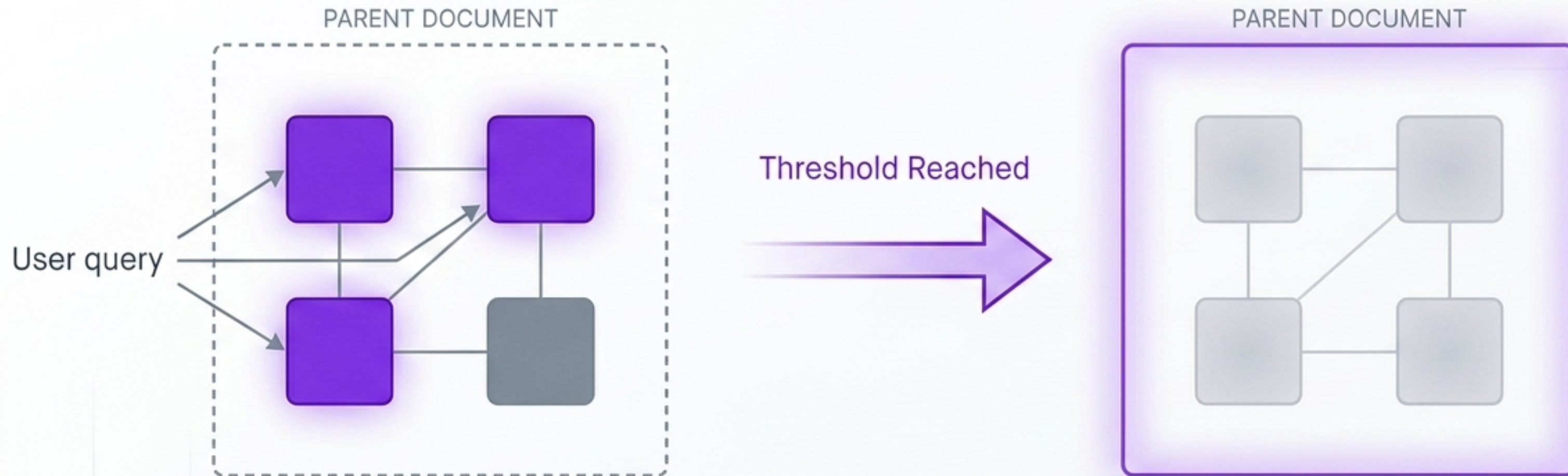
Target Use Case: Technical documentation, legal documents, and exact terminology requirements.

Document Summary Index Retrievers

A two-stage approach using summaries as a filter before returning full document content.



Auto-Merging Retriever: Preserving Context



Mechanism

Uses hierarchical chunking. If enough child nodes from the same parent are retrieved, it auto-merges and returns the parent node instead.

Architecture

Dual Storage system. Child chunks are stored for precise matching; parent chunks are stored for context retention.

Target Use Case

Long documents, legal papers, and technical specifications needing context preservation.

Recursive Retriever: Following the Thread

1 Core Capability

Follows relationships and references rather than just measuring top-down semantic similarity.

Node A



2 Support Mechanics

Handles lateral jumps through both chunk references and structured metadata references.

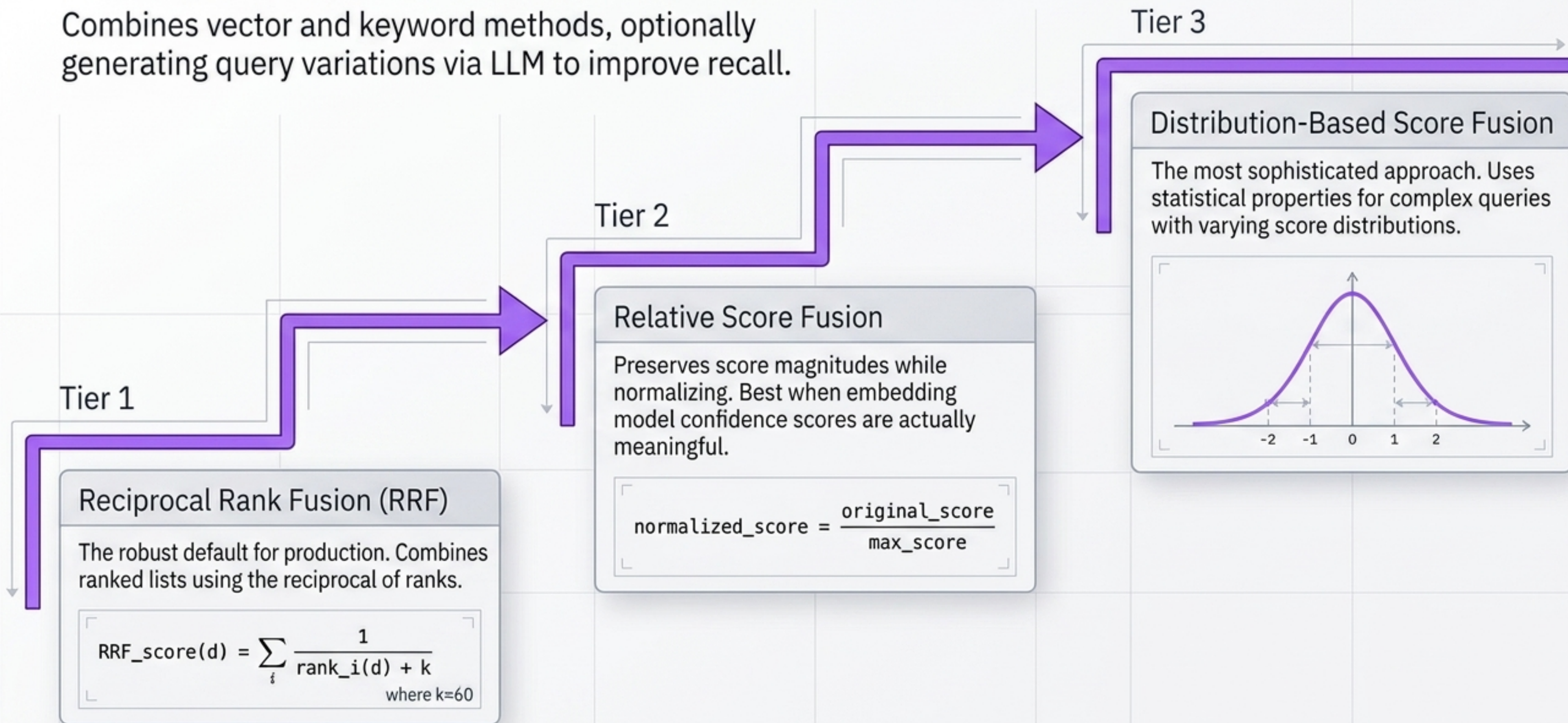
3 Ideal Deployment

Academic papers with citations, cross-referenced documentation, and interconnected knowledge bases.

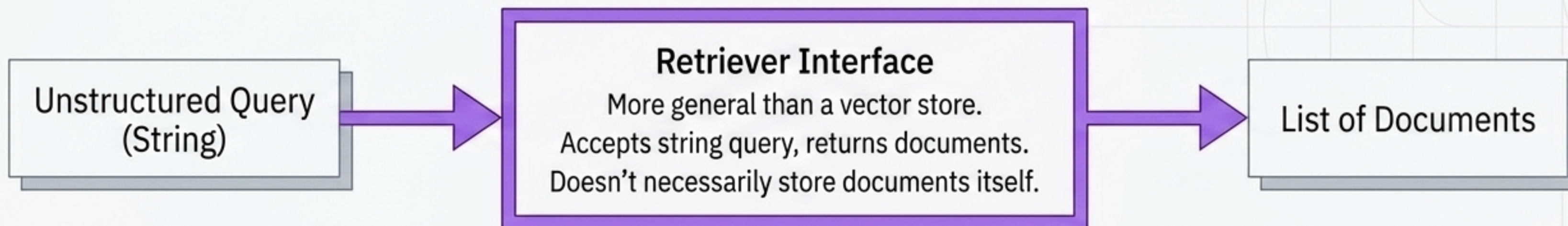
Node B

Query Fusion Retrievers: Combining Forces

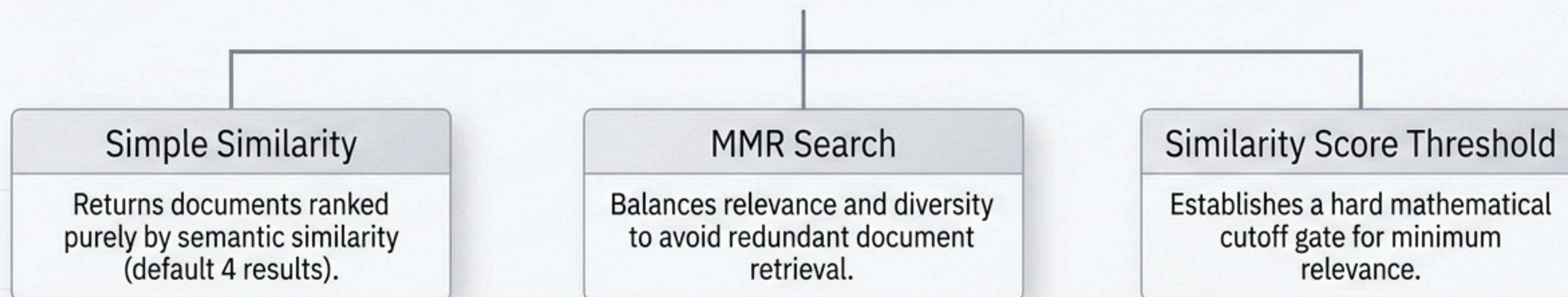
Combines vector and keyword methods, optionally generating query variations via LLM to improve recall.



The LangChain Paradigm: Query-First Retrieval

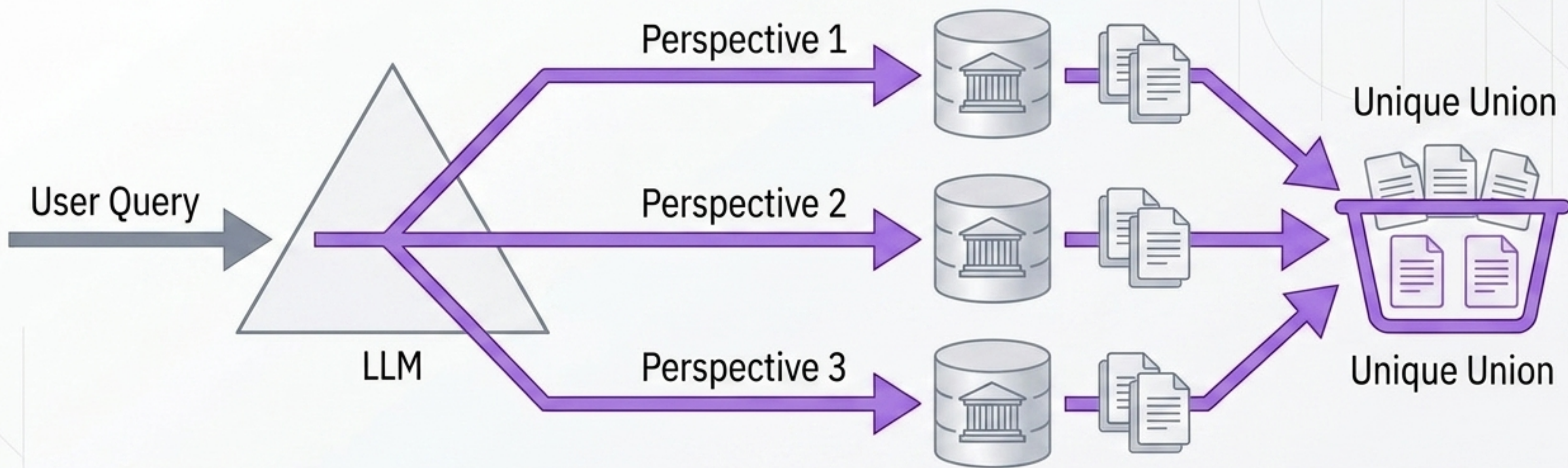


Foundation: Vector Store-Backed Retriever



Multi-Query Retriever: The Refraction Strategy

Problem: Distance-based vector database retrieval fails with subtle changes in query wording.

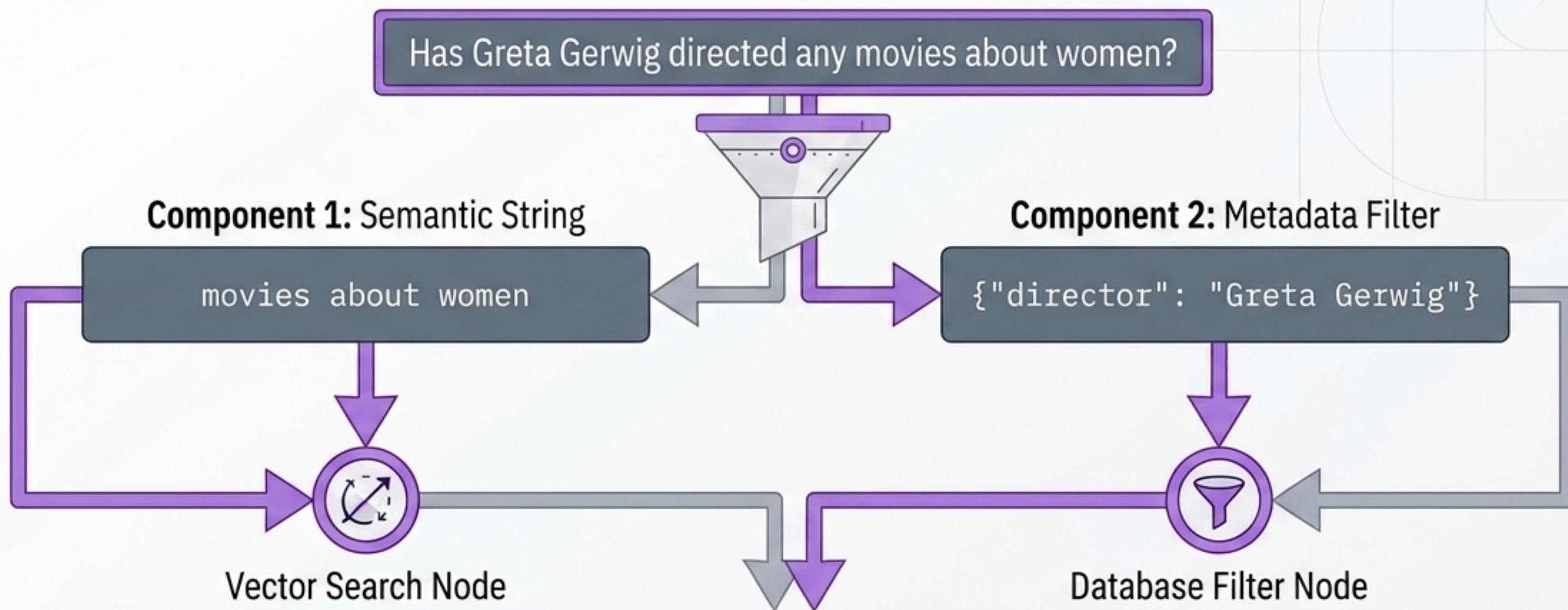


1. LLM generates multiple queries from different perspectives.

2. Retrieves relevant documents for each unique query independently.

3. Takes the unique union of all results to overcome distance-based limitations.

Self-Querying Retriever: Natural Language to Structured Logic

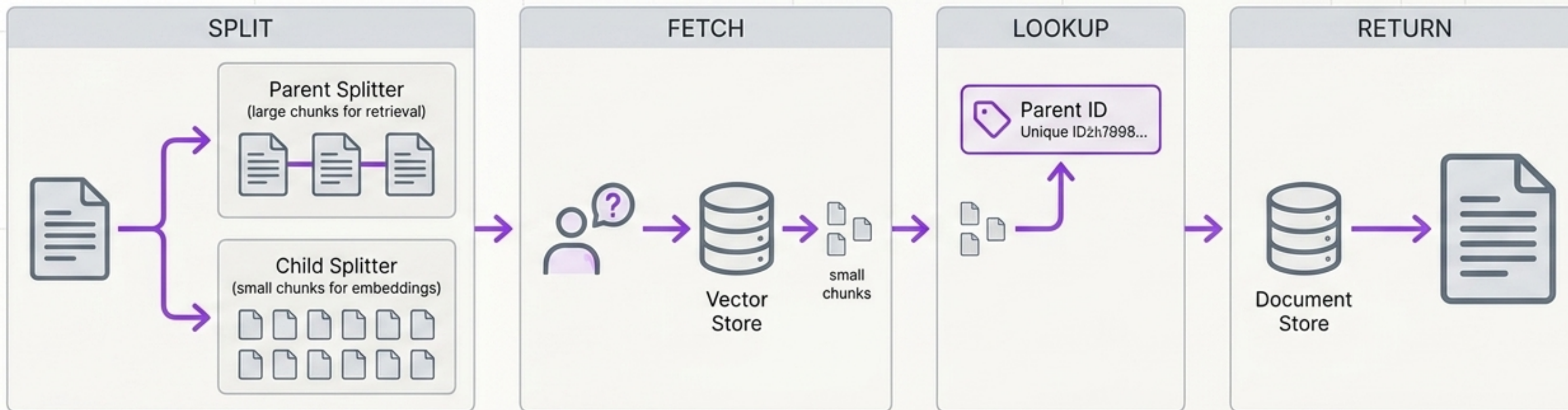


Requirement: Documents must possess rich, structured metadata with precise field descriptions.



Filter-Only Example: "I want to watch a movie rated higher than 8.5" (Triggers Metadata Filter without Semantic Search).

Parent Document Retriever: Resolving Conflicting Desires



The Conflict

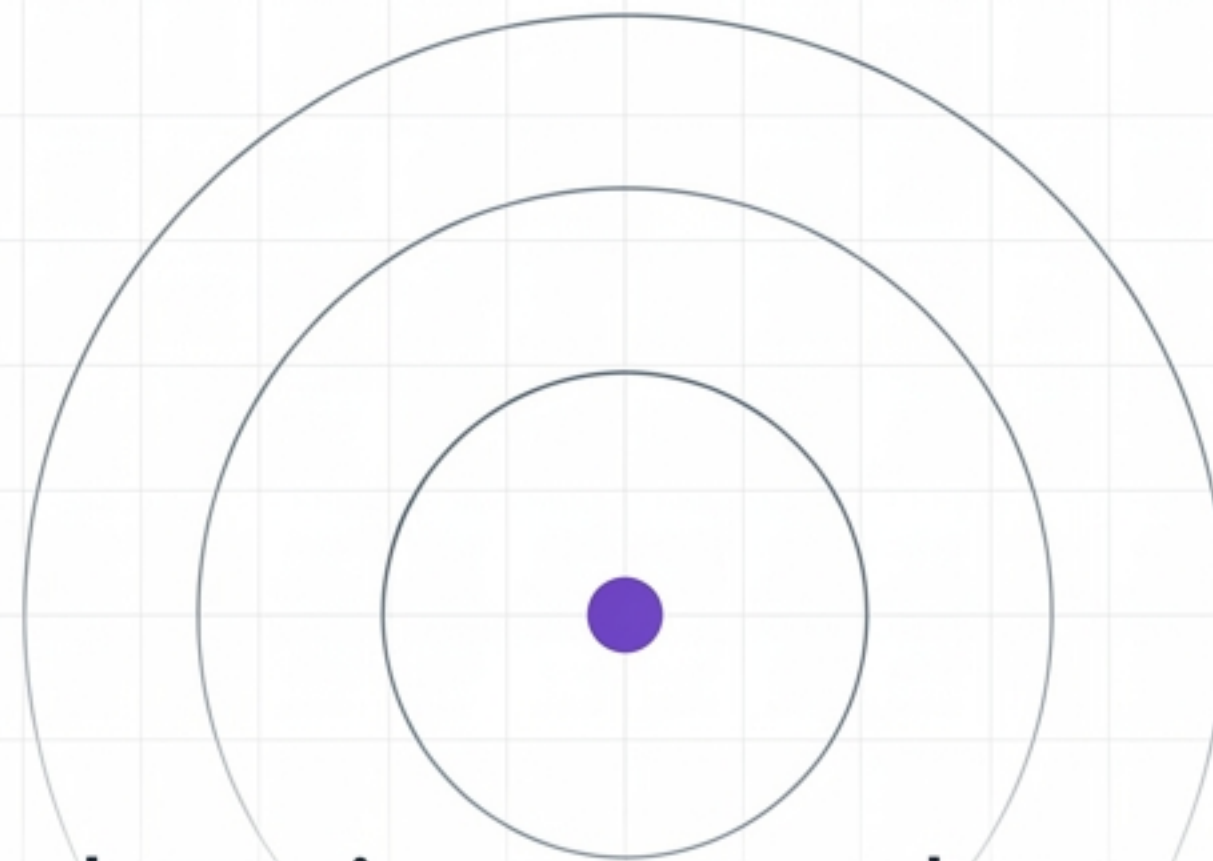
We need small documents for accurate embeddings, but large documents for context retention.

The Architecture

Dual splitters (parent/child) and dual storage mechanisms (vector store for embeddings, document store for parent documents).

The RAG Architect's Decision Framework

User Need	LlamaIndex Choice	LangChain Choice
Exact keyword matching	BM25 Retriever	Vector Store-Backed + custom logic
Multi-query with fusion	Query Fusion (RRF/Relative/Distribution)	Multi-Query Retriever (Union approach)
Citation following	Recursive Retriever	Not directly supported
Hierarchical context	Auto Merging Retriever	Parent Document Retriever
Simple semantic search	Vector Index Retriever	Vector Store-Backed Retriever



“Advanced retrievers go beyond simple similarity—they provide nuanced, context-aware information architecture.”